

Project 1: Web Application Penetration Testing (OWASP Top 10 Focus)

Week 1: Study of OWASP Top 10 (2025)

Rank	Category	Description	Key Focus in DVWA
A01	Broken Access Control	Users can act outside of their intended permissions (e.g., viewing other users' data). SSRF is now merged here.	Insecure Direct Object References (IDOR) and privilege escalation.
A02	Security Misconfiguration	Unhardened servers, default passwords, or open cloud storage. (Moved up from #5).	Default credentials (admin/password) and verbose error messages.
A03	Software Supply Chain Failures	Broadens "Vulnerable Components" to include compromised CI/CD pipelines and malicious dependencies.	Using outdated versions of Apache or PHP.
A04	Cryptographic Failures	Poorly encrypted data in transit or at rest.	Storing user passwords in cleartext or using weak MD5 hashes.
A05	Injection	Malicious code (SQL, Command, etc.) is executed by the server.	SQL Injection and Command Injection modules.
A06	Insecure Design	Architectural flaws that cannot be fixed by coding alone; requires threat modeling.	Flawed business logic (e.g., bypassing a checkout step).
A07	Authentication Failures	Failures in confirming user identity (brute force, weak passwords).	Brute Force module and session hijacking.
A08	Software or Data Integrity Failures	Failing to verify software updates or critical data before use.	Insecure deserialization of user input.

Rank	Category	Description	Key Focus in DVWA
A09	Security Logging & Alerting Failures	Inability to detect or respond to breaches due to lack of logs.	Checking if apache2 logs record your failed login attempts.
A10	Mishandling of Exceptional Conditions	(NEW) Poor error handling that leaks data or causes a "fail open" state.	Stack traces that reveal database structures when an error occurs.

Week 2: Test Vulnerability

1. SQL Injection

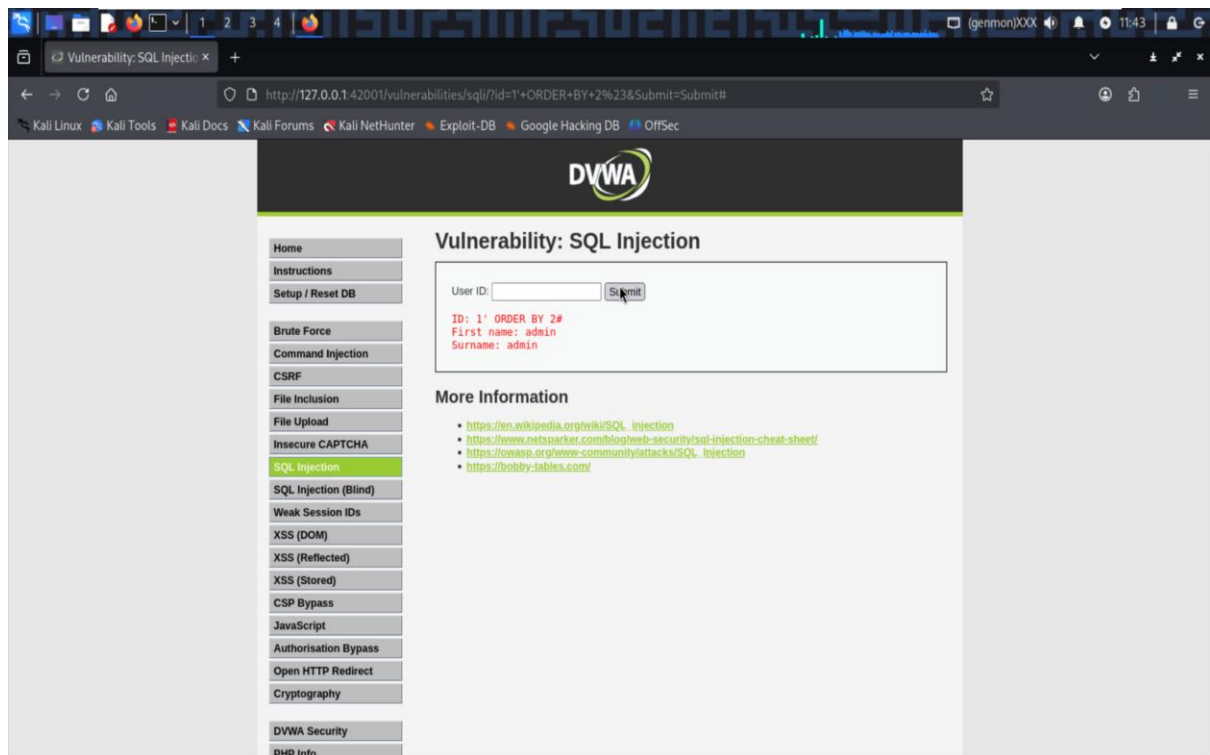
1. Confirm the Vulnerability

- Navigate to the **SQL Injection** tab in DVWA.
- In the User ID box, enter a single quote: '
- **Result:** You should see a MySQL error. This confirms the input is not sanitized and is being directly passed to the database.

2. Determine the Number of Columns

The UNION operator requires both queries to have the same number of columns.

- **Payload:** 1' ORDER BY 2# (No error means at least 2 columns exist).

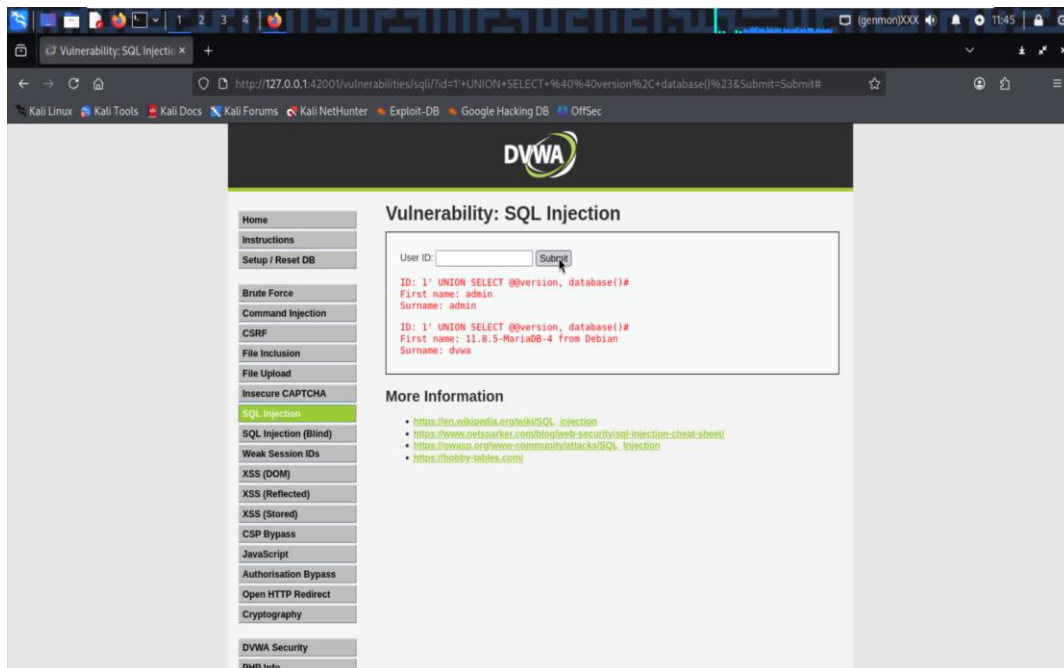


- **Payload:** 1' ORDER BY 3# (Error confirms there are exactly 2 columns).

3. Find the Database Name and Version

Now we use the UNION select to pull system information.

- **Payload:** 1' UNION SELECT @@version, database()#

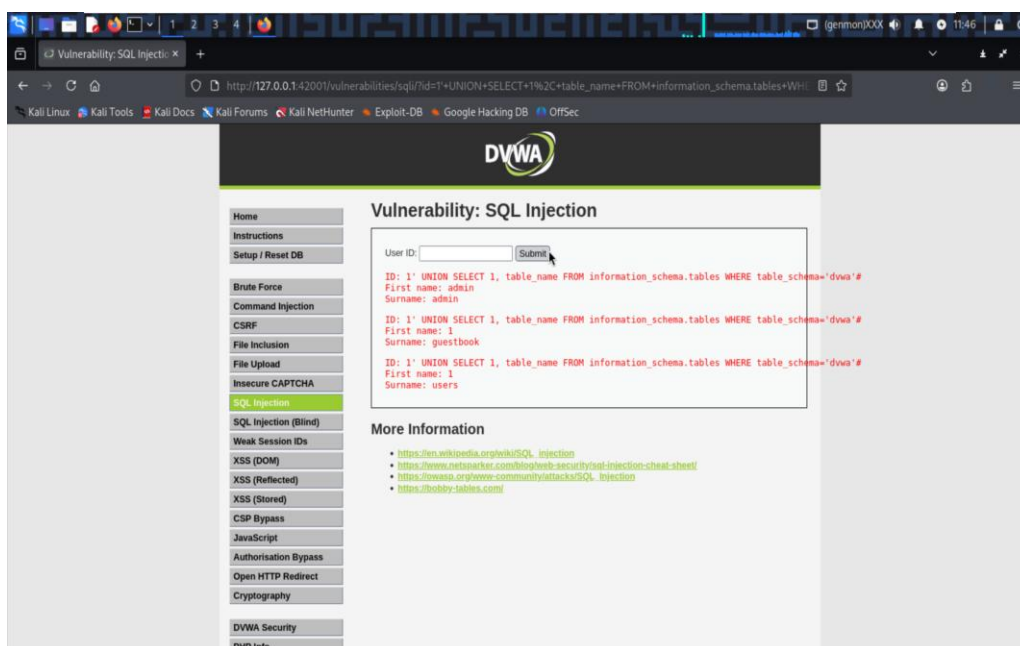


- **Result:** The page will display the MySQL version and the database name (usually dvwa).

4. Extract Table Names

We want to find where the users are stored.

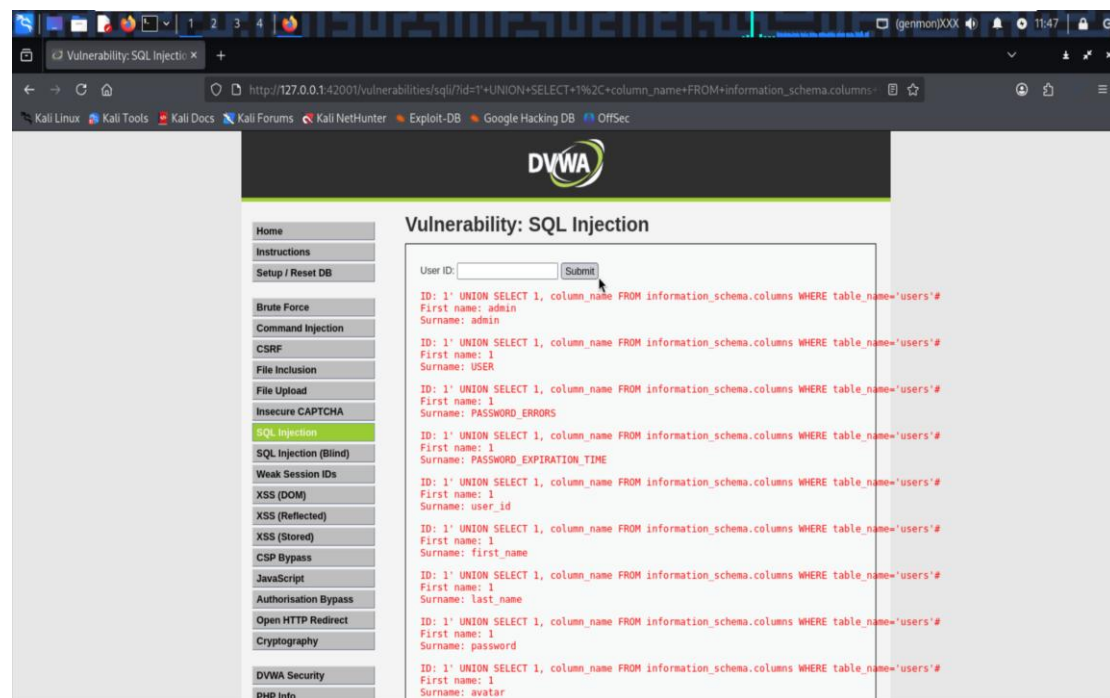
- **Payload:** 1' UNION SELECT 1, table_name FROM information_schema.tables WHERE table_schema='dvwa'#



- **Result:** You will see a list of tables, including one named users.

5. Extract Column Names from 'users'

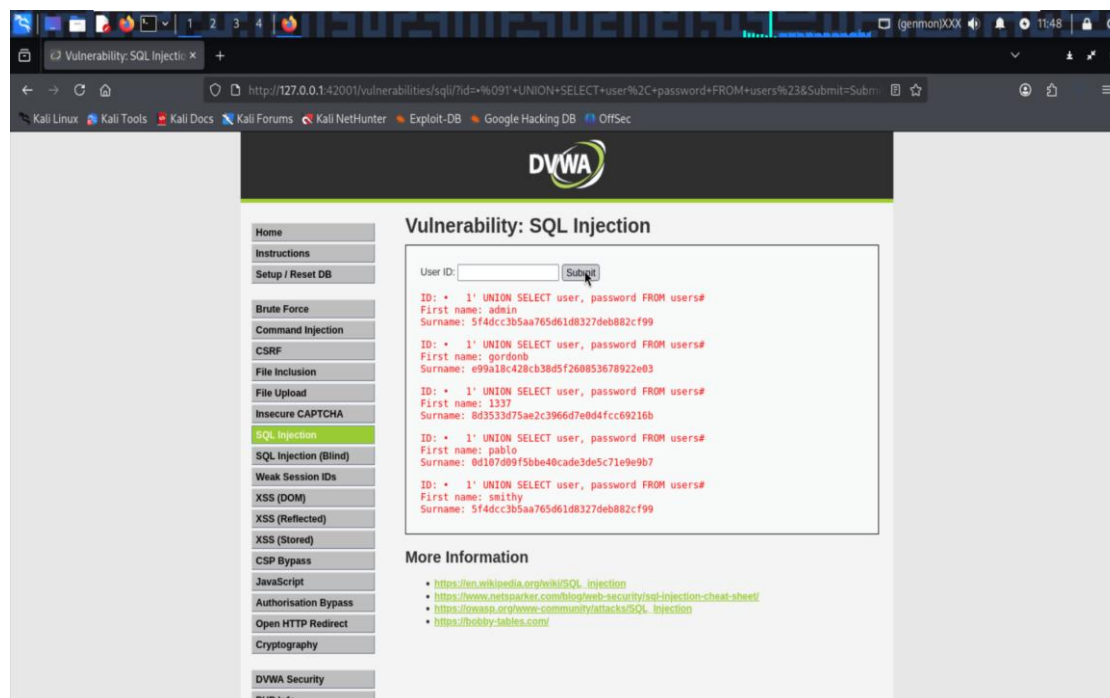
- **Payload:** `1' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name='users'#`



- **Result:** You'll see columns like user, password, first_name, and last_name.

6. Dump Usernames and Passwords

- **Payload:** `1' UNION SELECT user, password FROM users#`



- **Result:** You have now successfully dumped all usernames and their MD5 hashed passwords.

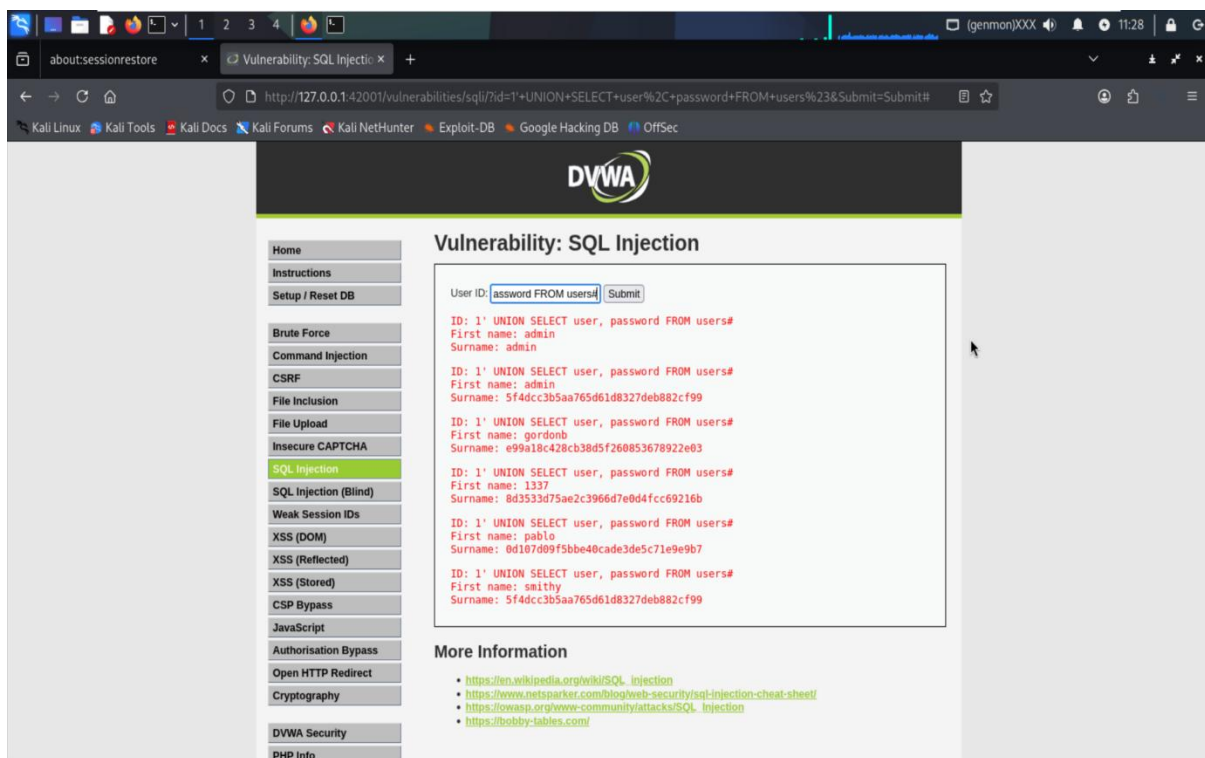
Proof-of-Concept (PoC) Report

1. Vulnerability Description

- **ID:** A05:2025 – Injection
- **Severity:** Critical
- **Description:** The "User ID" field in the SQL Injection module does not validate user-supplied data. This allows an attacker to append malicious SQL commands to the original query.

2. Proof of Concept (Step-by-Step)

1. **Objective:** Extract the administrative credentials from the backend database.
2. **Payload Used:** 1' UNION SELECT user, password FROM users#
3. **Resulting Query (Backend):**
\$\$SELECT \text{first_name, last_name FROM users WHERE user_id = '1'
UNION SELECT user, password FROM users\#';}\$\$
4. **Evidence:**



3. Impact

An attacker can gain full access to the database, leading to the exposure of sensitive user information, including encrypted passwords.

4. Remediation (Security Guidance)

To fix this, the developer should use **Prepared Statements (Parameterized Queries)**. This ensures that user input is treated as data, not as executable code.

Vulnerable Code (PHP):

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

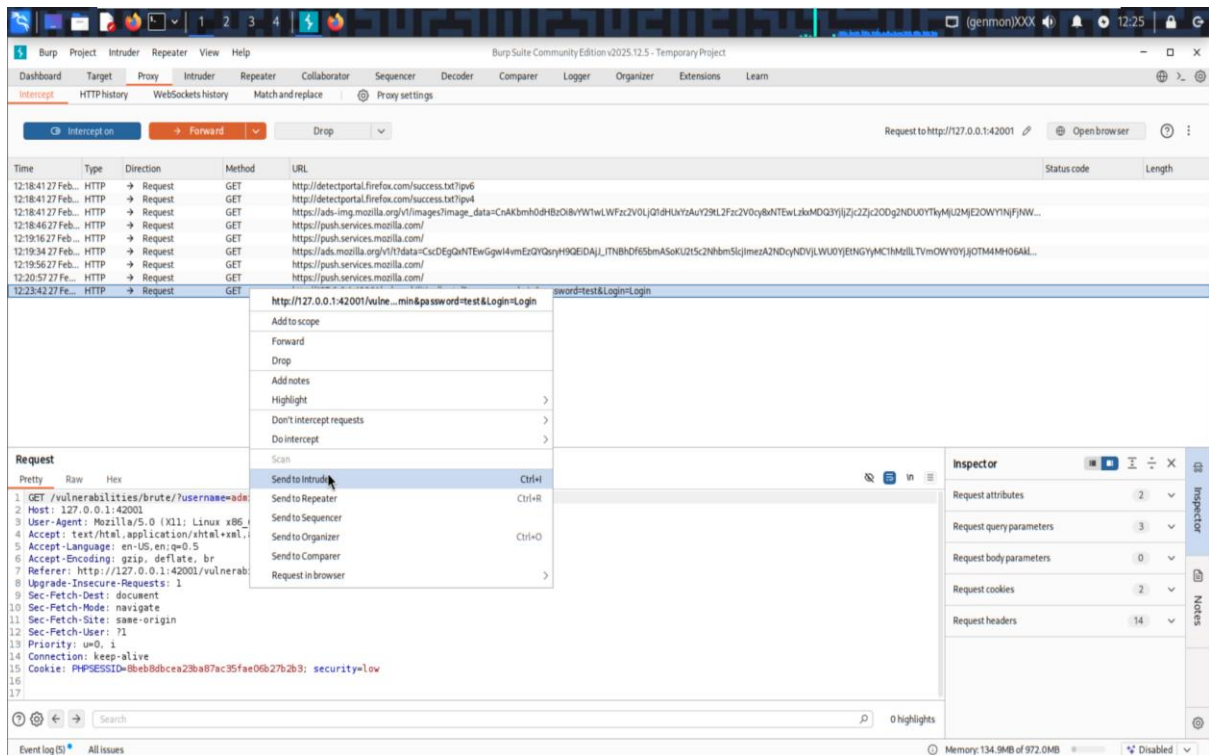
Secure Code (PHP/PDO):

```
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = :id');
$stmt->execute(['id' => $id]);
$user = $stmt->fetch();
```

2. Brute Force (Broken Authentication)

1. Capture the Request

- Open **Burp Suite** and ensure **Intercept is ON**.
- Go to the **Brute Force** tab in DVWA.
- Enter a random username and password (e.g., admin / test) and click **Login**.
- In Burp, you will see the GET request. Right-click the request and select "**Send to Intruder**".

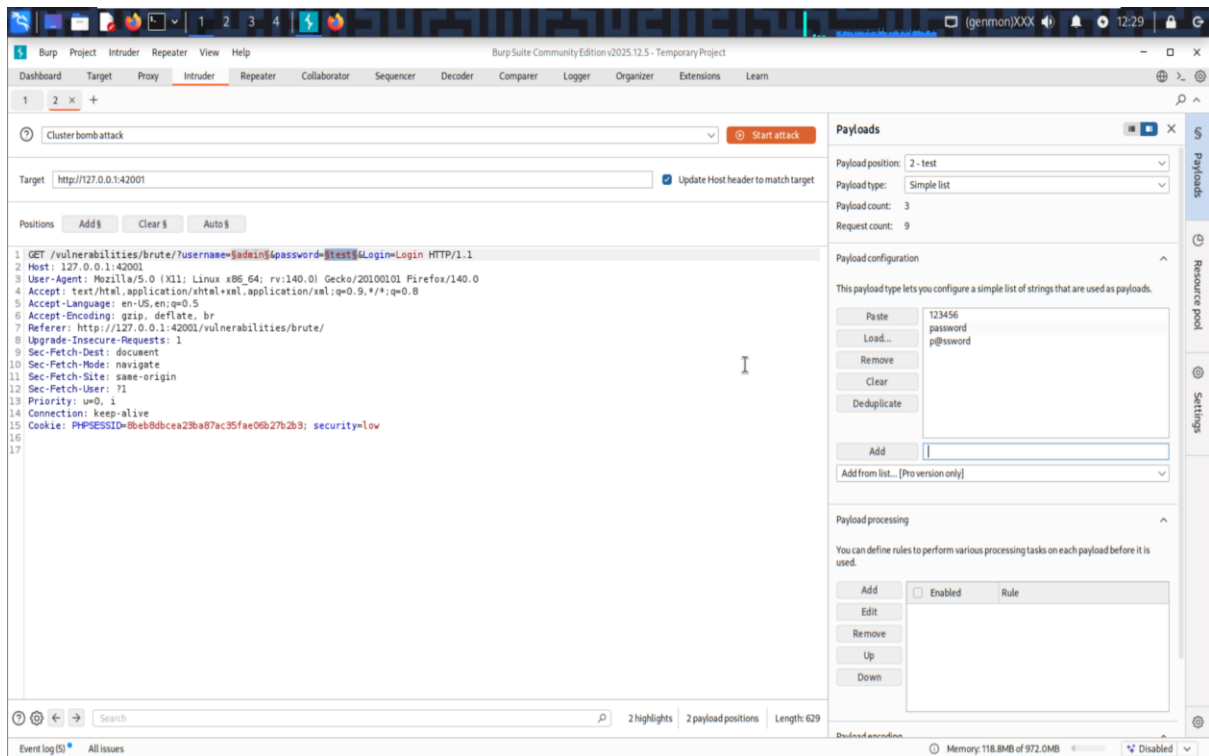


2. Set the Positions

- Go to the **Intruder > Positions** tab.
- You will see the URL parameters: username=admin&password=test&Login=Login.
- Click **Clear §** on the right.
- Highlight admin and click **Add §**. Highlight test and click **Add §**.
- Set the **Attack Type** to **Cluster Bomb** (this tests every combination of usernames and passwords).

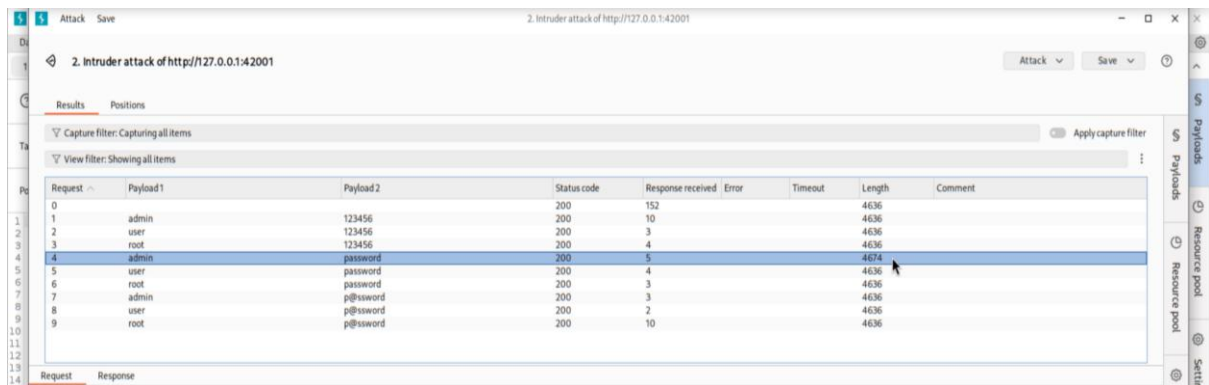
3. Load the Payloads

- Go to the **Payloads** tab.
- **Set 1 (Usernames):** Enter a few names like admin, user, root.
- **Set 2 (Passwords):** Load a small wordlist or type in common ones like 123456, password, p@ssword.

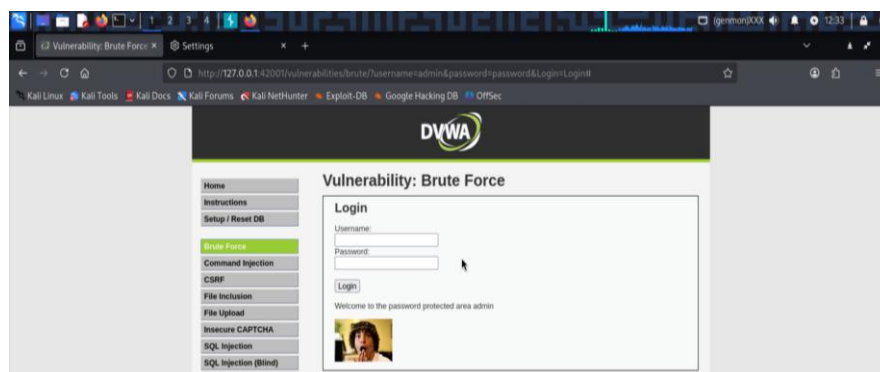


4. Launch the Attack

- Click **Start Attack**.
- **How to find the success:** Look for a request that has a different **Length** or **Status Code** (usually a 302 redirect or a different byte size) compared to the others.



- In DVWA, a successful login usually returns a smaller response length because it doesn't display the "Username/password incorrect" message.



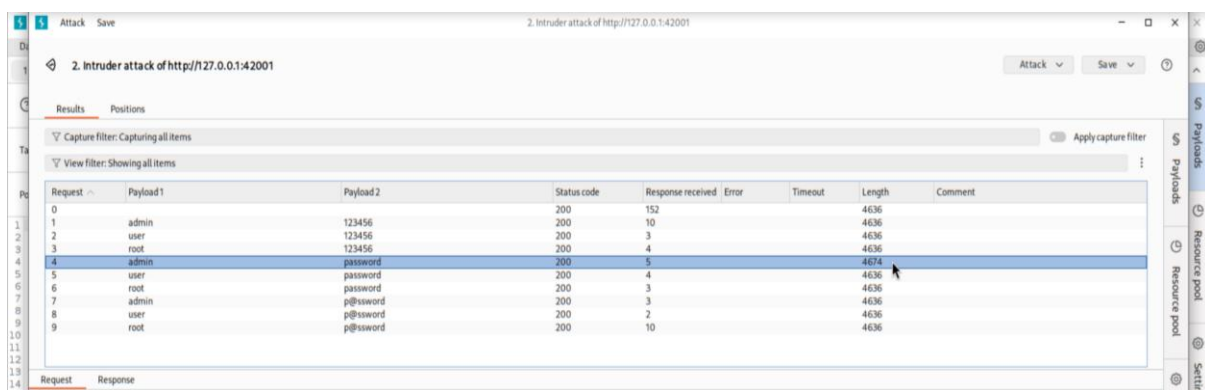
Proof-of-Concept (PoC) Report

1. Vulnerability Description

- **ID:** A07:2025 – Authentication Failures
- **Severity:** High
- **Description:** The login interface fails to implement mechanisms to prevent automated credential stuffing or brute-force attacks. There is no account lockout policy or CAPTCHA implementation.

2. Proof of Concept (Step-by-Step)

1. **Objective:** Gain unauthorized access to the admin account via automated password guessing.
2. **Tool Used:** Burp Suite Intruder.
3. **Attack Type:** Dictionary-based Brute Force.
4. **Evidence:**



The screenshot shows the Burp Suite Intruder interface with the title '2. Intruder attack of http://127.0.0.1:42001'. The 'Results' tab is active, displaying a table of attack results. The table has columns for Request, Payload 1, Payload 2, Status code, Response received, Error, Timeout, Length, and Comment. The results show a successful login for the admin user with the password 'password'.

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
0			200	152			4636	
1	admin	123456	200	10			4636	
2	user	123456	200	3			4636	
3	root	123456	200	4			4636	
4	admin	password	200	5			4636	
5	user	password	200	4			4636	
6	root	password	200	3			4636	
7	admin	p@ssword	200	3			4636	
8	user	p@ssword	200	2			4636	
9	root	p@ssword	200	10			4636	

5. **Result:** The password for user admin was identified as password.

3. Impact

An attacker can take over any user account given enough time and a robust wordlist. This leads to total account compromise and potential lateral movement within the application.

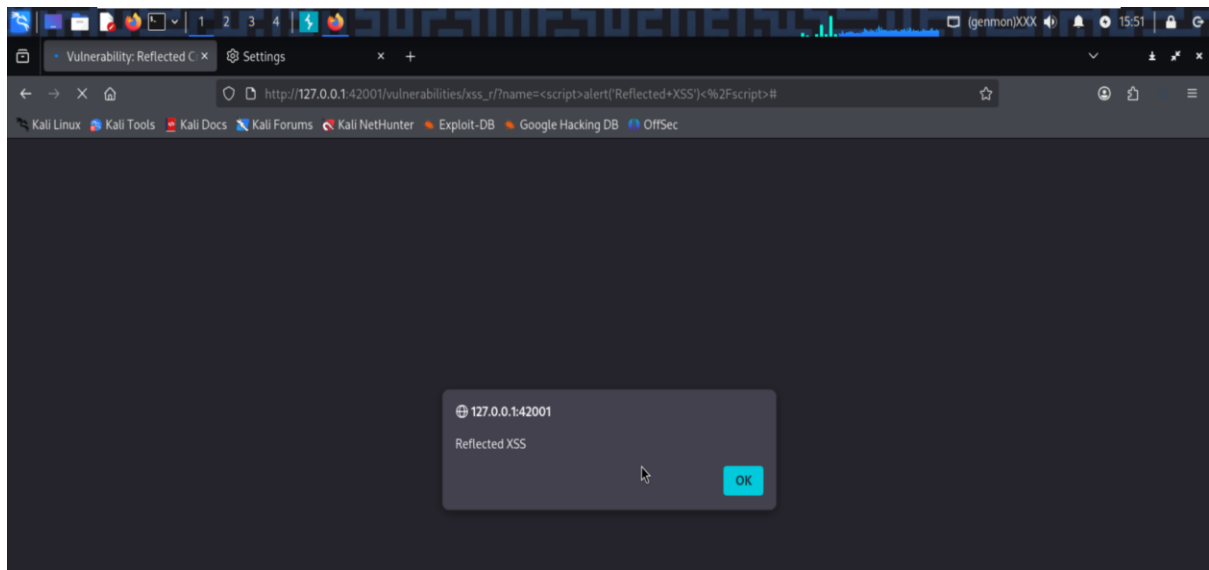
4. Remediation (Security Guidance)

- **Implement Rate Limiting:** Block or slow down requests from an IP after \$X\$ failed attempts.
- **Account Lockout:** Temporarily disable an account after 5 failed login attempts.
- **Multi-Factor Authentication (MFA):** Require a second form of verification.
- **Password Complexity:** Enforce strong password policies to make dictionary attacks harder.

3. The Three XSS Types

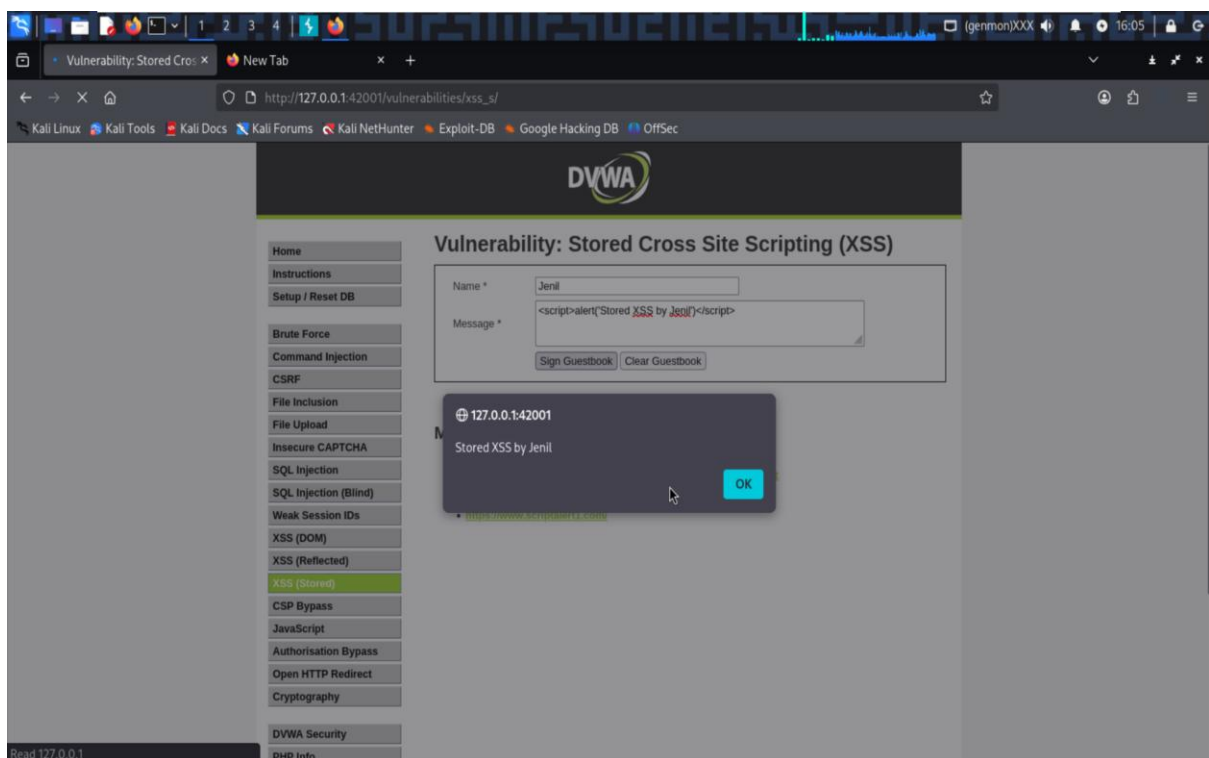
1. Reflected XSS (Non-Persistent)

- **Mechanism:** The malicious script is "reflected" off the web server to the victim's browser. It is usually delivered via a link (URL parameter).
- **DVWA Module:** XSS (Reflected)
- **Payload:** `<script>alert('Reflected XSS')</script>`
- **Proof of Concept:** Input the payload into the "Name" field. The URL becomes `?name=<script>....`. If you send this URL to another user, the script executes when they click it.



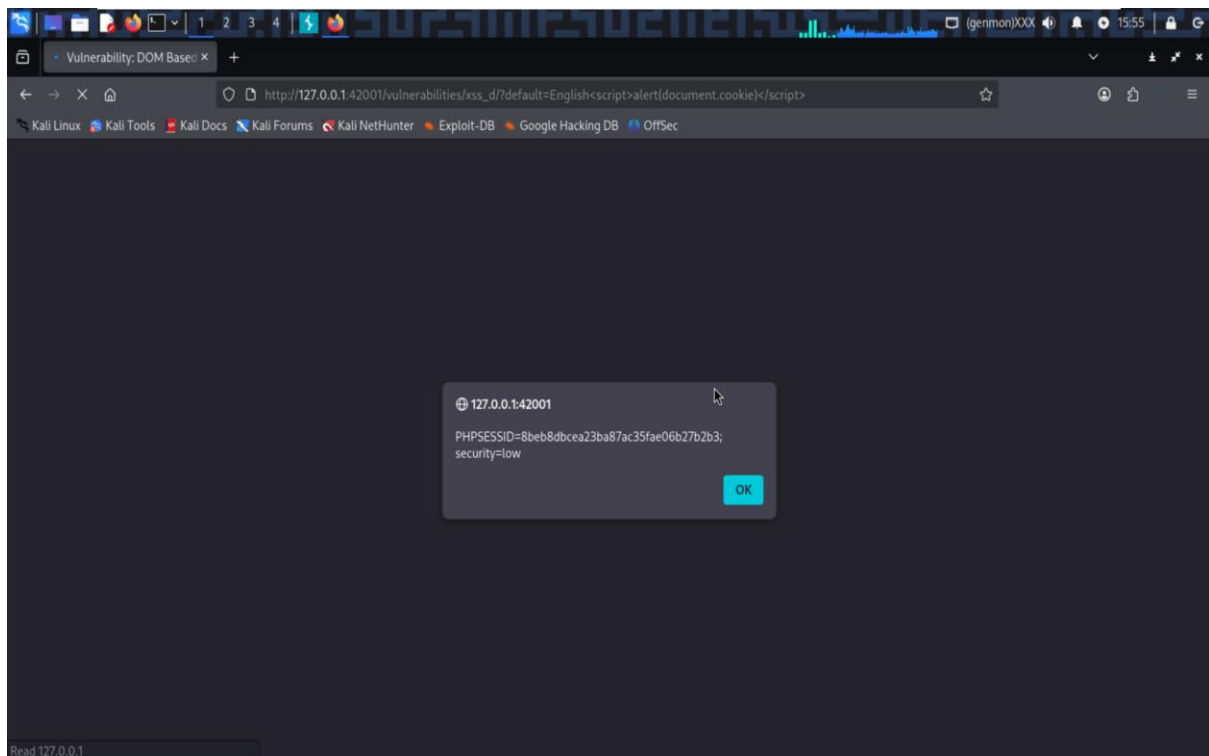
2. Stored XSS (Persistent)

- **Mechanism:** The script is permanently "stored" on the target server (e.g., in a database, comment section, or message board). Every user who visits the page will execute the script.
- **DVWA Module:** XSS (Stored)
- **Payload:** <script>alert('Stored XSS by Jenil')</script>
- **Proof of Concept:** Enter the payload into the "Message" box of the Guestbook. Now, every time you (or an admin) refreshes the Guestbook page, the browser attempts to send the session cookie to the attacker's server.



3. DOM-based XSS

- **Mechanism:** The vulnerability exists entirely in the **Client-Side** code. The server is never involved. The script execution happens when the page's client-side JavaScript reads data from a "source" (like the URL fragment) and passes it to a "sink" (like innerHTML).
- **DVWA Module:** XSS (DOM)
- **Payload:** Select a language, then change the URL manually to: `?default=English<script>alert(document.cookie)</script>`
- **Proof of Concept:** The script executes because the page's JavaScript reads the default parameter from the URL and writes it directly into the page DOM without validation.

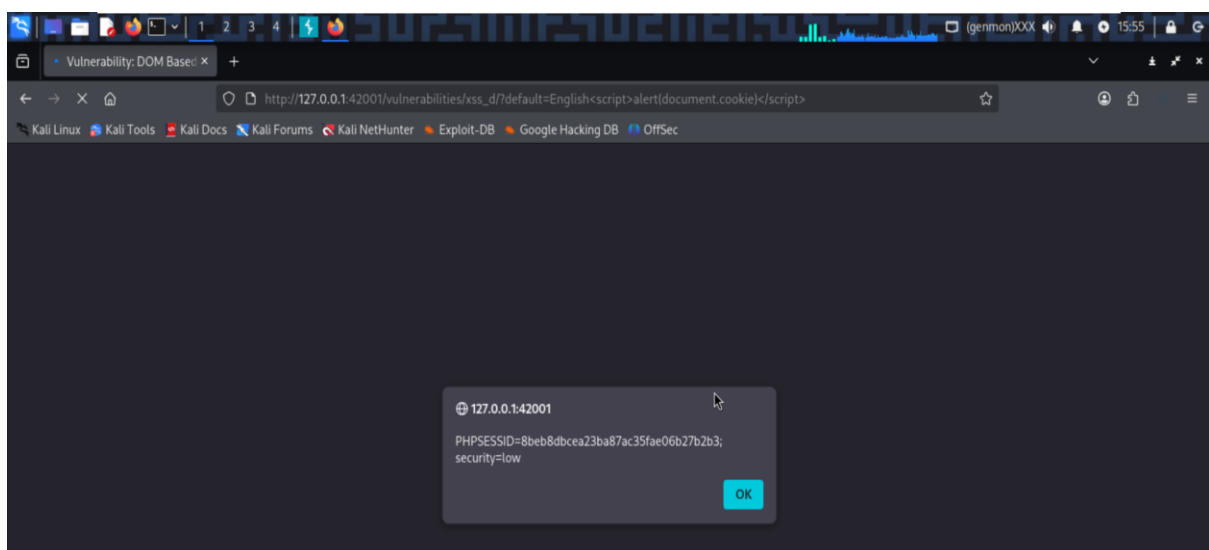
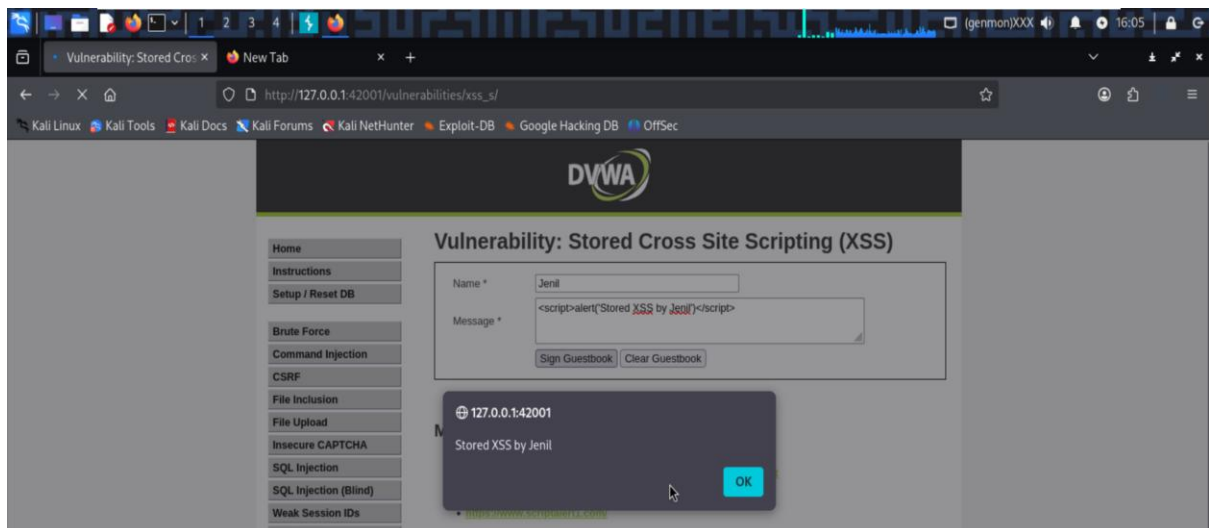
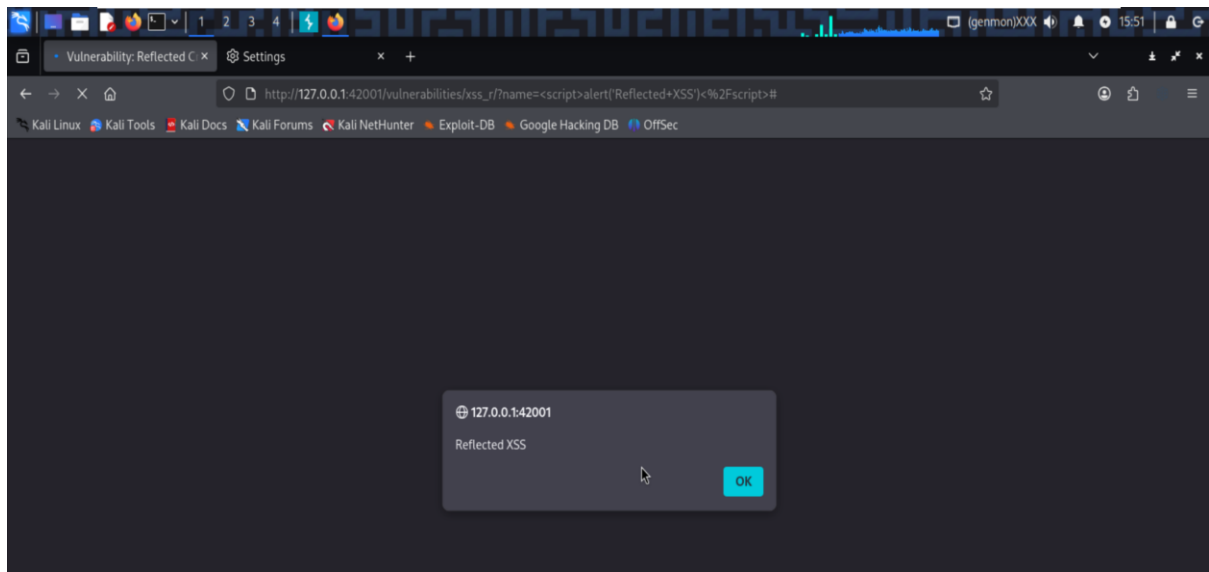


Proof-of-Concept (PoC) Report

Technical Summary Table

Type	Location of Payload	Persistence	Risk Level
Reflected	URL Parameter / Search Field	Temporary (Link-based)	High
Stored	Database / Guestbook	Permanent	Critical
DOM	Client-side URL Fragment	Browser-side only	High

Evidence & Screenshots



Impact Analysis

XSS allows for **Session Hijacking**, **Defacement**, and **Phishing**. In Stored XSS, a single payload can compromise every user who views a specific page, making it the most dangerous variant.

Remediation (The "Golden Rule")

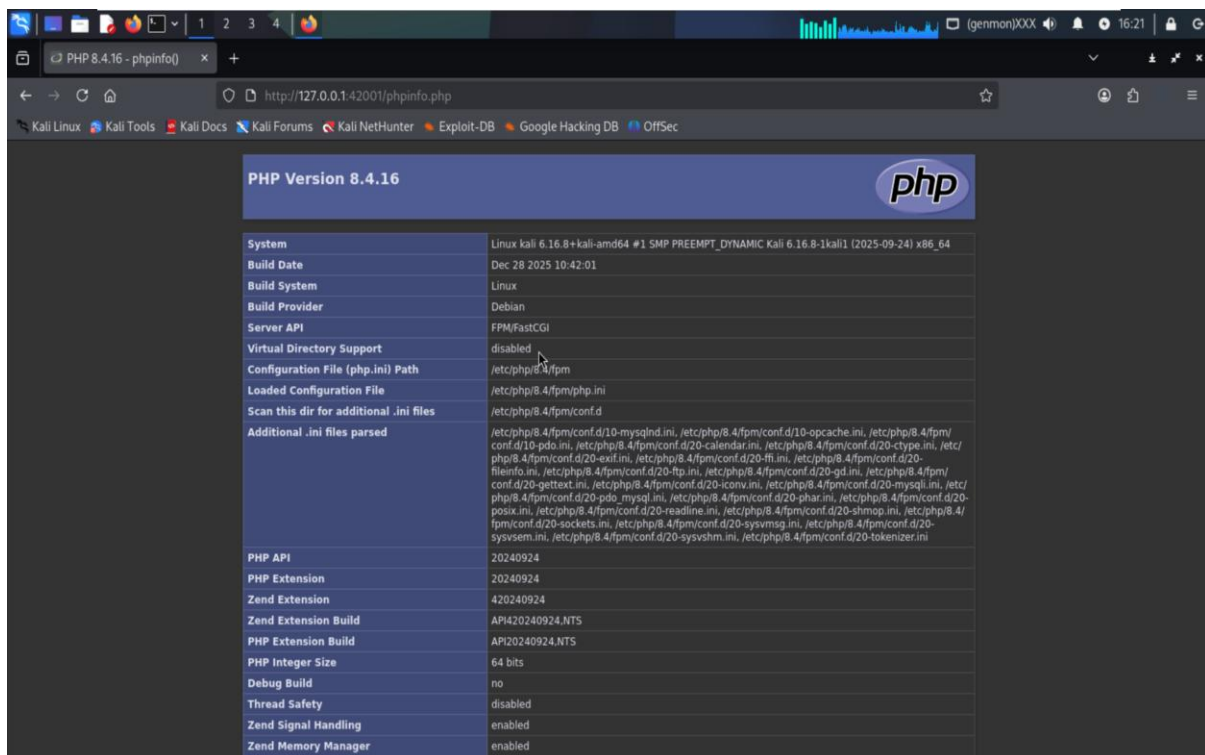
1. **Filter Input:** Validate against a whitelist of allowed characters.
2. **Encode Output:** Use context-aware encoding (e.g., htmlspecialchars() in PHP) so the browser treats <script> as plain text.
3. **Content Security Policy (CSP):** Implement a CSP header to restrict where scripts can be loaded from.

4. Sensitive Data Exposure

1. Information Leakage via phpinfo()

Many developers leave a phpinfo.php file on the server for debugging. This is a goldmine for attackers.

- **Action:** In your URL bar, try to navigate to: <http://localhost/phpinfo.php>
- **Result:** If accessible, you will see the exact version of PHP, OS details, loaded modules, and internal IP addresses.
- **Why it's a risk:** This helps an attacker choose specific exploits for the versions you are running.



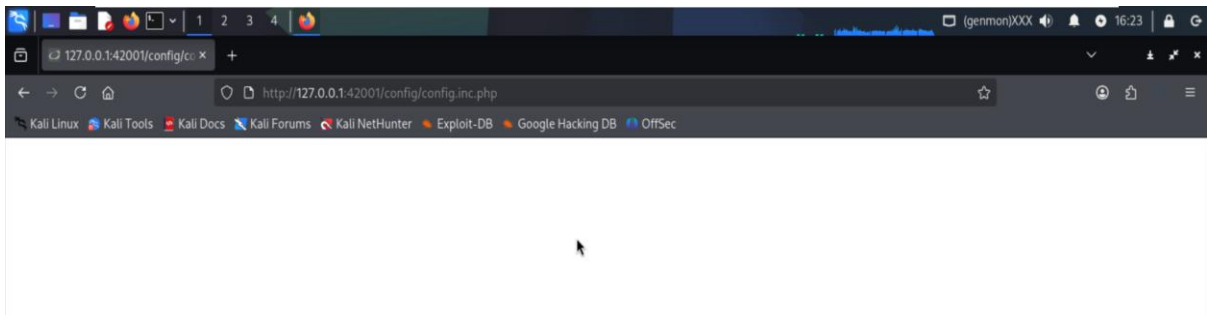
PHP Version 8.4.16	
System	Linux kali 6.16.8-kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24) x86_64
Build Date	Dec 28 2025 10:42:01
Build System	Linux
Build Provider	Debian
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.4/fpm
Loaded Configuration File	/etc/php/8.4/fpm/php.ini
Scan this dir for additional .ini files	/etc/php/8.4/fpm/conf.d
Additional .ini files parsed	/etc/php/8.4/fpm/conf.d/10-mysqlnd.ini, /etc/php/8.4/fpm/conf.d/10-opcache.ini, /etc/php/8.4/fpm/conf.d/10-pdo.ini, /etc/php/8.4/fpm/conf.d/20-calendar.ini, /etc/php/8.4/fpm/conf.d/20-ctype.ini, /etc/php/8.4/fpm/conf.d/20-exif.ini, /etc/php/8.4/fpm/conf.d/20-ftp.ini, /etc/php/8.4/fpm/conf.d/20-gd.ini, /etc/php/8.4/fpm/conf.d/20-gettext.ini, /etc/php/8.4/fpm/conf.d/20-iconv.ini, /etc/php/8.4/fpm/conf.d/20-mysqli.ini, /etc/php/8.4/fpm/conf.d/20-pdo_mysql.ini, /etc/php/8.4/fpm/conf.d/20-phar.ini, /etc/php/8.4/fpm/conf.d/20-posix.ini, /etc/php/8.4/fpm/conf.d/20-readline.ini, /etc/php/8.4/fpm/conf.d/20-shmop.ini, /etc/php/8.4/fpm/conf.d/20-sockets.ini, /etc/php/8.4/fpm/conf.d/20-sysvmsg.ini, /etc/php/8.4/fpm/conf.d/20-sysvsem.ini, /etc/php/8.4/fpm/conf.d/20-sysvshm.ini, /etc/php/8.4/fpm/conf.d/20-tokenizer.ini
PHP API	20240924
PHP Extension	20240924
Zend Extension	420240924
Zend Extension Build	API420240924.NTS
PHP Extension Build	API420240924.NTS
PHP Integer Size	64 bits
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled

2. Exposed Configuration Files

Sometimes, backup files or configuration files are left with weak permissions.

- **Action:** Try to access the database configuration file directly: <http://localhost/dvwa/config/config.inc.php>
- **Result:** Usually, the web server will execute the PHP and show a blank page. However, if the server misinterprets the file or if there is a .bak or .old version

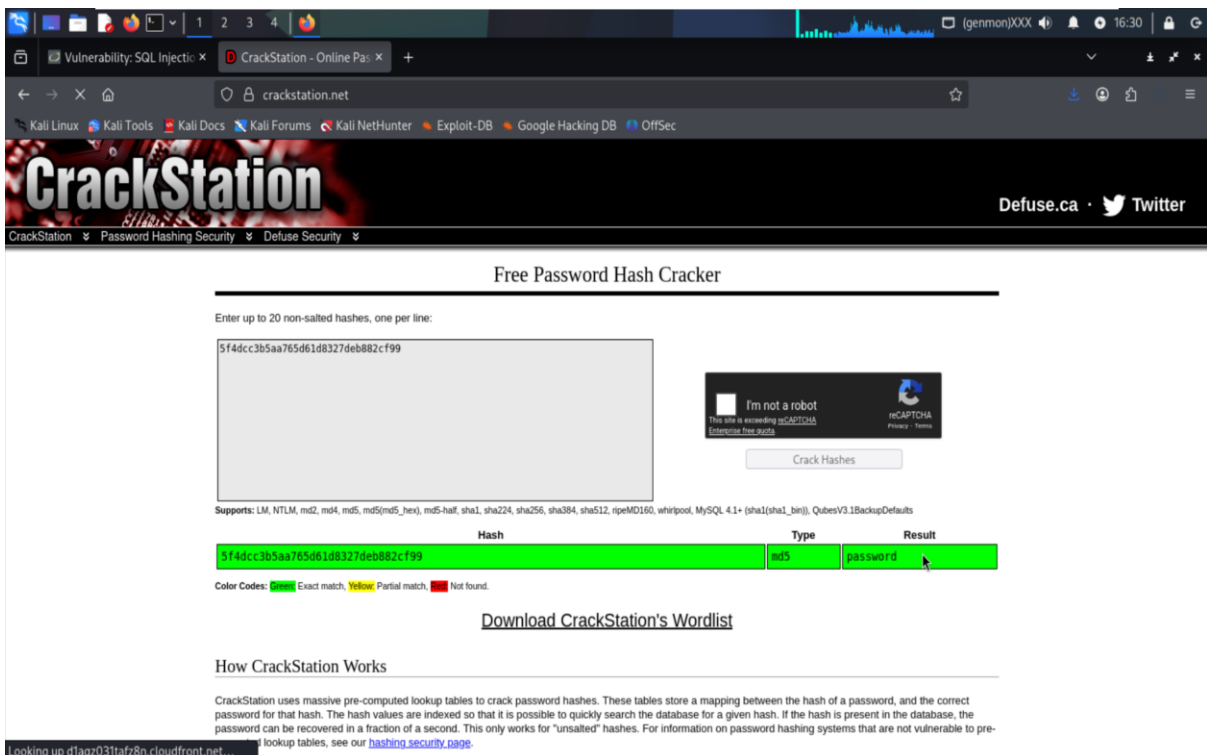
(e.g., config.inc.php.bak), the **database username and password** might be displayed in plain text.



3. Weak Password Hashing (The "Database Dump")

Recall the **SQL Injection** we did earlier. We extracted MD5 hashes.

- **The Exposure:** MD5 is considered "cryptographically broken."
- **Action:** Take a hash from your SQLi results (e.g., 5f4dcc3b5aa765d61d8327deb882cf99).
- **Test:** Put it into a site like *CrackStation*.
- **Result:** It instantly reveals the password is password. This proves the data was not "securely" stored.



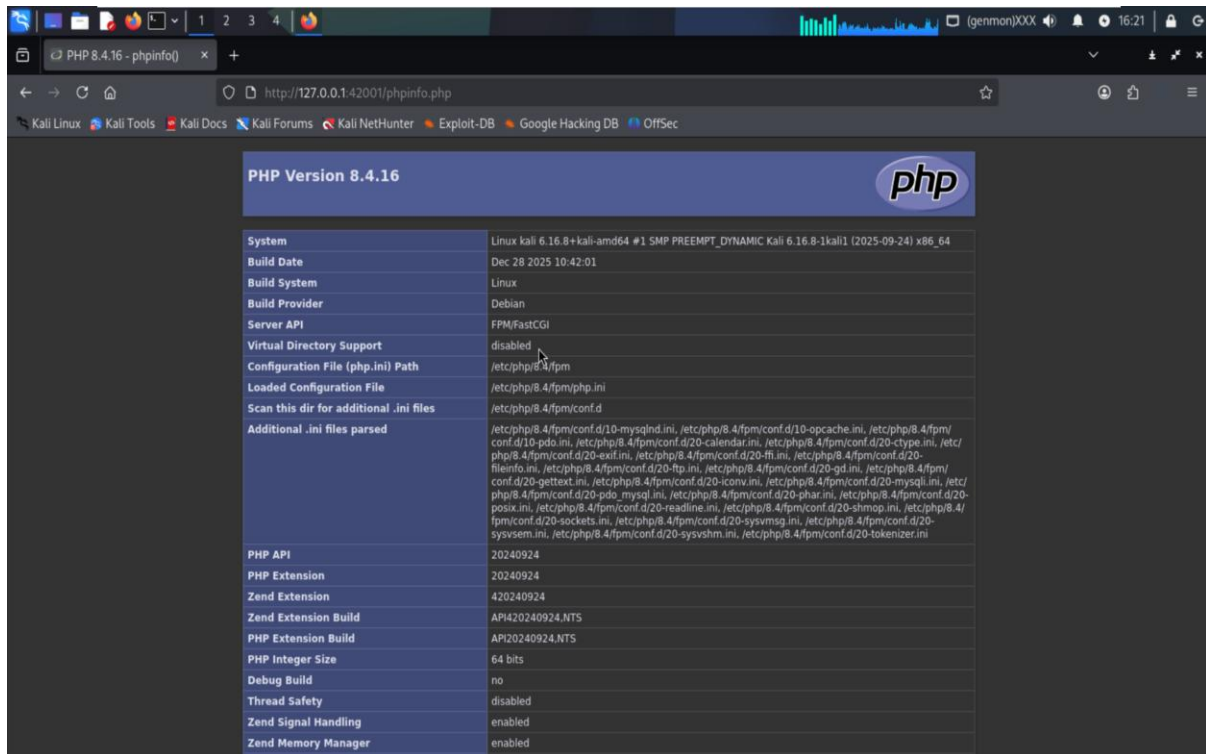
Proof-of-Concept (PoC) Report

1. Vulnerability Description

- **ID:** A04:2025 – Cryptographic Failures / Sensitive Data Exposure
- **Severity:** Medium to High
- **Description:** The application exposes system configuration details and utilizes weak cryptographic algorithms (MD5) for protecting sensitive user credentials.

2. Proof of Concept (Step-by-Step)

1. **Objective:** Identify system environment details and demonstrate the reversibility of stored credentials.
2. **Discovery 1:** Accessed `phpinfo.php`, revealing that the server is running **PHP Version 8.x** on **Apache/2.4.xx (Unix)**.
3. **Discovery 2:** Extracted user hashes via `SQLi`. Using a rainbow table attack, the MD5 hash `5f4dcc3b5aa765d61d8327deb882cf99` was cracked in less than 1 second.
4. **Evidence:**



PHP Version 8.4.16	
System	Linux kali 6.16.8+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24) x86_64
Build Date	Dec 28 2025 10:42:01
Build System	Linux
Build Provider	Debian
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.4/fpm
Loaded Configuration File	/etc/php/8.4/fpm/php.ini
Scan this dir for additional .ini files	/etc/php/8.4/fpm/conf.d
Additional .ini files parsed	/etc/php/8.4/fpm/conf.d/20-mysqld.ini, /etc/php/8.4/fpm/conf.d/20-opcache.ini, /etc/php/8.4/fpm/conf.d/20-pdo.ini, /etc/php/8.4/fpm/conf.d/20-calendar.ini, /etc/php/8.4/fpm/conf.d/20-ctype.ini, /etc/php/8.4/fpm/conf.d/20-exif.ini, /etc/php/8.4/fpm/conf.d/20-ftp.ini, /etc/php/8.4/fpm/conf.d/20-gd.ini, /etc/php/8.4/fpm/conf.d/20-gettext.ini, /etc/php/8.4/fpm/conf.d/20-iconv.ini, /etc/php/8.4/fpm/conf.d/20-mysqli.ini, /etc/php/8.4/fpm/conf.d/20-pdo-mysql.ini, /etc/php/8.4/fpm/conf.d/20-phar.ini, /etc/php/8.4/fpm/conf.d/20-posix.ini, /etc/php/8.4/fpm/conf.d/20-readline.ini, /etc/php/8.4/fpm/conf.d/20-shmop.ini, /etc/php/8.4/fpm/conf.d/20-sockets.ini, /etc/php/8.4/fpm/conf.d/20-sysvmsg.ini, /etc/php/8.4/fpm/conf.d/20-sysvsem.ini, /etc/php/8.4/fpm/conf.d/20-sysvshm.ini, /etc/php/8.4/fpm/conf.d/20-tokenizer.ini
PHP API	20240924
PHP Extension	20240924
Zend Extension	420240924
Zend Extension Build	API20240924.NTS
PHP Extension Build	API20240924.NTS
PHP Integer Size	64 bits
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled

3. Impact

Attackers can use version information to find known CVEs (Common Vulnerabilities and Exposures) for the specific software running. The use of MD5 allows attackers to perform offline cracking of the entire user database if leaked.

4. Remediation (Security Guidance)

- **Disable/Remove Debug Files:** Remove `phpinfo.php` and any `.bak`, `.sql`, or `.git` files from the production web root.
- **Strong Hashing:** Use modern, slow hashing algorithms like **Argon2** or **bcrypt** with a unique salt for every user.
- **Encrypt Data in Transit:** Ensure all sensitive pages are served over **HTTPS** (TLS 1.3).